Decisiones arquitectónicas
TP6 - Grupo 10

Repositorio: https://github.com/LucasSalina/2026_ICS_Grupo10

User Story «Inscribirme a actividad» de EcoHarmony Park.

Las rutas de código son relativas a `Programa/`.

Plataforma y estructura

1. Se implementa como una aplicación web responsive utilizando el Framework Django

2. **Estructura monolítica**, sin SPA ni API REST. Todo en una sola app (`core`).

3. Para el formulario solo se utiliza HTML y el servidor, porque la consulta de turnos tiene un botón de submit propio que se oculta cuando hay JS. `inscripcion.js` solo agiliza la carga y no tiene lógica de negocio.

4. **SQLite como base de datos** (`actividades.sqlite3`). Los datos se cargan con comandos de gestión idempotentes (`get_or_create`): `cargar_actividades` arma el catálogo y `cargar_horarios` genera turnos para 7 días abiertos desde hoy.

5. **Sin login.** Tanto la sesión y el usuario quedan simulados.

6. **Deploy en Render con configuración por entorno**. `settings.py` lee `DJANGO_SECRET_KEY` y `DJANGO_DEBUG` de variables de entorno y conserva los valores de desarrollo por defecto; gunicorn sirve la aplicación y whitenoise los estáticos. Como el disco del plan gratuito no persiste, cada arranque migra la base y vuelve a cargar los turnos.

Modelo de dominio

1. Las inscripciones se registran para uno o mas participantes (`Inscripcion` **1:N** `Participante`)

2. **El cupo disponible se calcula, no se guarda.** `cupos_disponibles()` resta los participantes inscriptos a la capacidad de la actividad.

3. **El cupo cuenta participantes, no inscripciones**. Esto con el fin de evitar sobreventas

5. **Reglas de datos separadas de la persistencia.** `core/inscripciones.py` tiene la validación sin acceso a la base (`validar_solicitud`). El modelo valida lo que necesita consultar la base: cupo, horario y simultaneidad.

6. **Los datos de entrada viajan en un dataclass inmutable** (`DatosParticipante`, `frozen=True`) entre la vista y el dominio.

7. **Una excepción específica por regla de negocio**, todas subclases de `ValueError`. (Ej: `CupoInsuficienteError`, `TalleObligatorioError` e `InscripcionSimultaneaError`) La vista las captura juntas y muestra el mensaje en el formulario. Un horario inexistente es la excepción: devuelve 404.

8. **Validación de edad en un componente aislado y reutilizado** (`validacion_edad.py`), invocado por cada participante.

9. **Constantes de dominio compartidas.** Cupos (`CUPO_MAXIMO_POR_ACTIVIDAD`), actividades con talle, talles válidos y edades mínimas se definen una sola vez y los usan modelos, vistas y los comandos de carga.

10. **Escritura atómica.** La inscripción y sus participantes se guardan en `transaction.atomic()`. Las reglas de negocio se validan antes de escribir; si la escritura falla a mitad de camino, no queda una inscripción con participantes a medias.

Reglas de tiempo

1. **Zona horaria del parque** `America/Argentina/Cordoba`. Los turnos se interpretan en hora local, no en UTC.

2. **Los dos límites de la ventana de reserva son inclusivos:** se puede reservar hasta 24 h antes y hasta 15 min antes del turno (`<=`).

3. **El calendario está en constantes de código, no en la base:** lunes, 1/1 y 25/12 cerrados, y turnos de 9 a 17.

Correo y QR

1. **Correo y QR son adaptadores separados.** `email.py` envía el correo y `qr.py` genera la imagen con segno.

2. **Un fallo del envío no cancela la inscripción.** Se registra en el log y la confirmación avisa que el correo no salió.

3. **El QR codifica solo un código de referencia** (`EcoHarmony-Inscripcion-{pk}`), sin DNI ni correo.

4. **Dos canales de envío elegidos por configuración**. Si existe `BREVO_API_KEY`, el correo sale por la API HTTP de Brevo, porque Render bloquea el SMTP saliente en el plan gratuito; si no, por SMTP, que sirve para desarrollo local. Por SMTP el QR va incrustado por `cid`; Brevo no lo admite, así que ahí viaja como adjunto y el correo lo avisa. La clave, el remitente y el servidor SMTP se leen de `settings`.

5. **Sin precios ni pagos.** Se quitaron explícitamente porque pertenecen a la otra User Story.

Presentación

1. **El formulario de inscripción se lee a mano en la vista** (`_leer_participantes`, con campos `participante_N_*`), sin Django Forms ni formsets.

2. **Solo se muestran los cupos disponibles**, no la capacidad máxima. La lista de turnos excluye los que no tienen lugar.

3. **Los textos de términos y descripciones son constantes en** `views.py`, no datos de la base.

4. **Fuente Lato servida localmente** y paleta del FAQ, con diseño pensado primero para el celular.

Pruebas y calidad

1. **Pruebas en pytest/pytest-django** y las variantes de una regla están parametrizadas.

2. **Pruebas aisladas de servicios externos y del reloj.** El SMTP y la API de Brevo se simulan para toda la suite con fixtures `autouse` en `conftest.py` y la hora con `patch("django.utils.timezone.now")`.

3. **Formato y tipos obligatorios con Black, Ruff y mypy**, y dependencias con versión fijada en `requirements*.txt`.

Fuera de alcance

1. **No hay concurrencia, locks ni idempotencia**, ni reintentos del correo, y el esquema no tiene restricción de unicidad para `Horario`. Por ahora, que no haya turnos duplicados depende solo del `get_or_create` de `cargar_horarios`.